

Code-Layout, Readability and Reusability

Date	30 June 2026
Team ID	XXXXXX
Project Name	Rising Waters
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation strictly followed across all Python and React source files.
2	Proper File Structure	Files and folders are logically organized	Yes	Clean separation with dedicated directories for components and backend ML APIs.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Explicit naming used throughout the project like <code>water_level_metrics</code> .
4	Function / Method Names	Functions are descriptively named	Yes	Descriptively named functions like <code>calculateFloodRisk()</code> and <code>fetch_sensor_data()</code> .
5	Code Comments	Inline and block comments explain logic	Yes	Comprehensive inline docstrings explain complex mathematical steps in ML logic.
6	Modular Design	Code is split into reusable functions/modules	Yes	Codebase is fully broken down into lightweight standalone API microservices.

8	Error Handling	Exceptions and errors are handled gracefully	Yes	Robust try-except blocks intercept bad data feeds or broken API requests gracefully.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared functionalities are extracted into common utility files to avoid duplication.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Weather Data Preprocessor	Python / Pandas / NumPy	Reused for both model training and real-time prediction input preprocessing.	High
2	Flood Prediction Engine	Python / Scikit-learn / Random Forest / XGBoost	Reused across model evaluation, testing, and Flask application predictions	High
3	Feature Validation Module	Python	Reused for validating rainfall, cloud visibility, and seasonal rainfall inputs before prediction.	High
4	Prediction Results Interface	HTML5 / CSS3 / Flask	Reused for displaying flood risk predictions and user interaction across all prediction pages.	Medium
5	Model Loader & Inference Module	Python / Flask	Reused for loading the trained model and generating predictions for every user request.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Highly structured layout with clean architecture layers, adhering perfectly to standard software engineering patterns.
Readability	5	Variables and explicit functions explain their purpose clearly, eliminating any ambiguous code segments.
Reusability	4	Core telemetry parsers and warning engines are modularly engineered for easy deployment in other disaster setups.
Documentation / Comments	5	Exhaustive API endpoint descriptions and markdown readmes simplify future upgrades or system maintenance protocols.
Overall Score	4.75	Excellent overall code quality demonstrating strong industry-level practices, ready for production deployment.